inramkernel in the MovieCart STM32 firmware

Oct 18, 2026 · @Bill

`inramkernel` is true while the 6507 is running the RAM kernel out of RIOT, the roughly 60-line stretch of each frame when the cart is idle and the STM32 is free for SD work. Every wait in the firmware now comes down to one rule: outside relaxed mode, serve the bus until `inramkernel` goes false → true, then do the next step. In relaxed mode (playback field loads), leave the bus alone and assume the whole list of steps finishes inside the current window.

This describes the uncommitted changes in `firmware/stm32` as of this date. Nothing here has been built with the ARM toolchain or run on hardware; the edited files pass a clang syntax check, and the host test (Host test results) runs the boot and playback code against FAT32 images and an emulated 6507.

The flag

The STM32 sees every address the 6507 puts on the bus. Cart reads (A12 high) go through `bus_dispatch()`; on every other cycle (A12 low) each bus loop calls `mc_track_ram_kernel(addr)` from `core.h`. The flag changes on the exact RIOT-RAM fetch where the 6507 switches kernels.

Bus event	What the STM32 sees	Effect
ColdStart `jmp $0080`	Cart read of `$F053` (jmp high byte)	Sets `mc_ram_kernel_armed` to `MC_ARMED_COLDSTART`
First RAM kernel opcode after ColdStart	A12-low read of `$0080` while armed	`inramkernel = true`, disarm, set `mc_coldstart_entry`, set up the next visible kernel
OsHead `RTS` (end of the 6 cart overscan lines)	Cart read of `$F1C5`	Sets `mc_ram_kernel_armed` to `MC_ARMED_FRAME`
RTS stack reads	`$01FD`–`$01FF`	Ignored (A8 set, not RIOT RAM)

Bus event	What the STM32 sees	Effect
RTS throwaway read	`$0082`	Ignored as an entry (`$0082` only ever clears)
First RAM kernel opcode after RTS	`$0083` while armed	`inramkernel = true` , disarm, clear `mc_coldstart_entry` , set up the next visible kernel
RAM kernel `jsr VisibleBars` , last RAM cycle	`$0082` (jsr high byte)	`inramkernel = false`
VisibleBars entry	Cart read of `$F09D`	Nothing beyond serving the byte

The tracker only matches RIOT RAM `$0080` – `$00FF` with A8 low, so ColdStart's copy loop (which writes `$80` – `$D1` before the arm) and the stack never trigger it. If the exact entry cycle is missed, the next RIOT-RAM read sets the flag a few cycles late. Nothing backs up the `$0082` clear any more: if it is missed, the flag stays true through the visible kernel and the next frame wait catches the following entry instead, losing one frame. That can only happen if relaxed work overruns the window.

On each entry the tracker also calls `mc_on_ram_kernel_entry()` , which sets up the next visible kernel from the displayed buffer: the picture pointers (`graphBuf` , `colorBuf` , `colorBKBuf`) and audio pointer back to its start, the line count and the loop target. The visible kernel advances the pointers every line pair and nothing else resets them, so without this a frame with no field swap would start where the last one ended and eventually read past the end of SRAM; with it, such a frame repeats the last field. This runs on a free cycle with the whole window ahead, and a swap later in the same window overrides it. The cart-read handlers at `$F09D` and `$F1C5` now do nothing beyond serving the byte, except one store at `$F1C5` (`mc_ram_kernel_armed`).

ColdStart's entry is the exception to the long window: its first RAM kernel instruction is the `jsr VisibleBars` itself, about 5 bus cycles before `$F09D` . `mc_coldstart_entry` marks it, and `mc_blanking_swap()` skips the field swap after it; the boot check in `bus_wait_endframe()` skips it as well.

Bus primitives

One serve loop, `serve_until_ram_kernel()` in `bus_service.c`, does all the waiting: it serves cart reads and returns on the false → true edge of `inramkernel`. Called while already in the RAM kernel, it waits for the next frame's entry. Everything else is built on it.

Function	Relaxed (`mc_sdio_gate_relaxed`)	Not relaxed	Title field swap
`wait_ram_kernel() (static)`	Waits for the edge	Waits for the edge	No
`moviecart_wait_ram_kernel_entry()`	Waits for the edge	Waits for the edge	Yes, on entry
`moviecart_bus_yield()`	Returns, bus untouched	Same as `moviecart_wait_ram_kernel_entry()`	Yes, when it waits
`moviecart_delay_ms(ms)`	Waits for entries until the deadline; one check per frame	Same	Yes
`bus_wait_endframe(timeout)`	Waits for the first entry after `$F1C5`, with a timeout	Same	Yes
`emulate_cartridge()`	Loops forever: wait, swap	Same	Yes, every frame
`bus_serve_ms()`	Serves for a wall-clock time (LED codes), tracks the flag only	Same	No

The title swap is `mc_blanking_swap()` in `core.c`: it calls `mc_field_swap_to_display()` unless `mc_playback_pipeline` is set. During playback the frame waits therefore never swap; the only swap is in `finishFieldRead()`, once a field has loaded and passed its checks (see Frame reads). A frame with no swap, including every frame spent in `bus_serve_ms()`, shows the last field again.

Removed in this change: `moviecart_sdio_gate()` (the SDIO driver calls `moviecart_bus_yield()` instead), `moviecart_wait_blanking()`, `moviecart_bus_pump()`, `bus_serve_cycle()`, `mc_sd_strict`, `mc_swap_pending`, `mc_blanking_window`, `mc_visible_bars_vended`, the `$F09D` / `$F1C5` audio and line-count work, the frame-length diagnostic, the heartbeat, `PUMP_ALIGN`, the `GAP_PROBE` build, the seek checkpoint queue and table (replaced by the extent map), and the `fatalStrobe()` scratch-read diagnostic. `moviecart_wait_blanking_start()` was renamed `moviecart_wait_ram_kernel_entry()`.

SD card initialization

Card init and mount run with relaxed mode off, so every SDIO step starts at the beginning of its own RAM kernel window: one command, one register write or one status poll per frame.

1. **Boot check.** `main()` calls `wait_title_sync(3000)`, which is `bus_wait_endframe()`. It skips ColdStart's entry and waits for the first entry after `$F1C5`, proving a whole frame ran. No frame in 3 s → `fatalBlink(6)`.
2. **Title hold.** 120 frame waits (`TITLE_HOLD_FRAMES`, about 2 s), each swapping the title field.
3. **Mount.** `setupDisk()` waits for a fresh entry, then `pf_mount()` → `mc_disk_initialize()` → `TM_FATFS_SD_SDIO_disk_initialize()`. A failed mount → `fatalBlink(5)`.
4. **`SD_Init()`.** Power-on, card identification, `SDIO_Init` at 400 kHz then the transfer clock, card info, select, and the 4-bit bus. Every command goes through `SDIO_SendCommandGated()` (a yield, then the command) and its response poll (`CmdResp*Error`) yields before each status check, so each command costs at least two frames. Up to 5 attempts, with `moviecart_delay_ms(50)` between them (about 3 frames).
5. **Boot sector and FAT setup.** `pf_mount()` reads the boot sector with `disk_read_block1()`, which yields once (a fresh entry) before reading and keeps the sector in an 8-slot LRU cache, and calls `moviecart_wait_ram_kernel_entry()` between each parsing step.

Every one of these waits ends in a title field swap, so the title keeps alternating through steps 3–5.

Opening the file

Opening and selecting files also run with relaxed mode off: each directory sector, cluster lookup and sector read starts on a fresh RAM kernel entry.

At boot (`setupDisk()`, after `stage_pulse(2)`):

1. Wait for a fresh entry, then `pf_open_file(&numFrames, 1)` walks the root directory one 32-byte entry at a time. Each entry costs three frame waits (`moviecart_wait_ram_kernel_entry()`, a yield after the read, and one more before the next entry); the sector itself comes from the cache after the first entry. Only movie

files count: 8.3 extension `MVC`, at least one field long, and a name not starting with `.` or `_`, so macOS `.DS_Store` and `._*` AppleDouble files are skipped. No movie file → `fatalBlink(8)`.

2. `pf_map_file()` maps the file's clusters (see FAT lookups under the risks). Then two more fresh-entry waits around `stage_pulse(3)`, then `updateInit()`.
3. `runTitle()` reads field 1's header sector through the sector cache (`title_probe_work()` → `disk_try_read_block1()`) to learn the file's geometry. A failed read → `fatalBlink(10)`; a wrong header → `fatalBlink(7)`. A field 1 past the end of the file → `fatalBlink(9)`.

File select during playback (`checkSelectVideo()`, on Select or end of file). Playback has `mc_playback_pipeline` set, so none of these waits swap a title field.

Step	Yields	Approx. frames
`pf_open_file()` for the next file, wrapping to file 1	Three frame waits per directory entry, plus SDIO command yields for new sectors	3 per entry; a few seconds for a later file in a directory of long names
Map the new file's clusters (`pf_map_file()`)	One async CMD18 of 6 FAT sectors per frame	1 per 768 clusters (~35 for an hour of video on 32 KB clusters)
Geometry probe of field 1 (`disk_try_read_block1()`; read failure → code 10)	Read waits and SDIO yields	A handful
`clearFieldBuffers()`, 2 × 3,072 bytes in 64-byte chunks	One yield per chunk	48
Settle	30 × `waitEndFrame()`	30

Selecting a file therefore takes about 2 s or more before playback restarts, depending on how far the directory walk goes and how large the movie is. Until `clearFieldBuffers()` runs, the last field stays on screen and its audio repeats as a buzz.

LED blinking

Milestone blinks are timed by counting RAM kernel entries; failure blinks are timed by the wall clock and ignore `inramkernel`, so they stay countable even when the 6507 has lost sync.

Blink	Function	Timed by	Where the LED changes	Title swaps
Milestones 1–5 (`stage_pulse(n)`)	`moviecart_stage_blink()`	`moviecart_wait_ram_kernel_entry()`: 60 entries rest, then per flash 21 on and 20 off, then 60 rest (141 frames, about 2.4 s, for one flash)	Right after a RAM kernel entry	Every frame, except milestone 5 (playback already set up)
Failure codes (`fatalBlink(code)`), forever	`flash_led_slow()` → `bus_serve_ms()`	DWT: 1.2 s lead-in, 300 ms on / 400 ms off per flash, 1.2 s tail	Wherever the wall clock expires	None

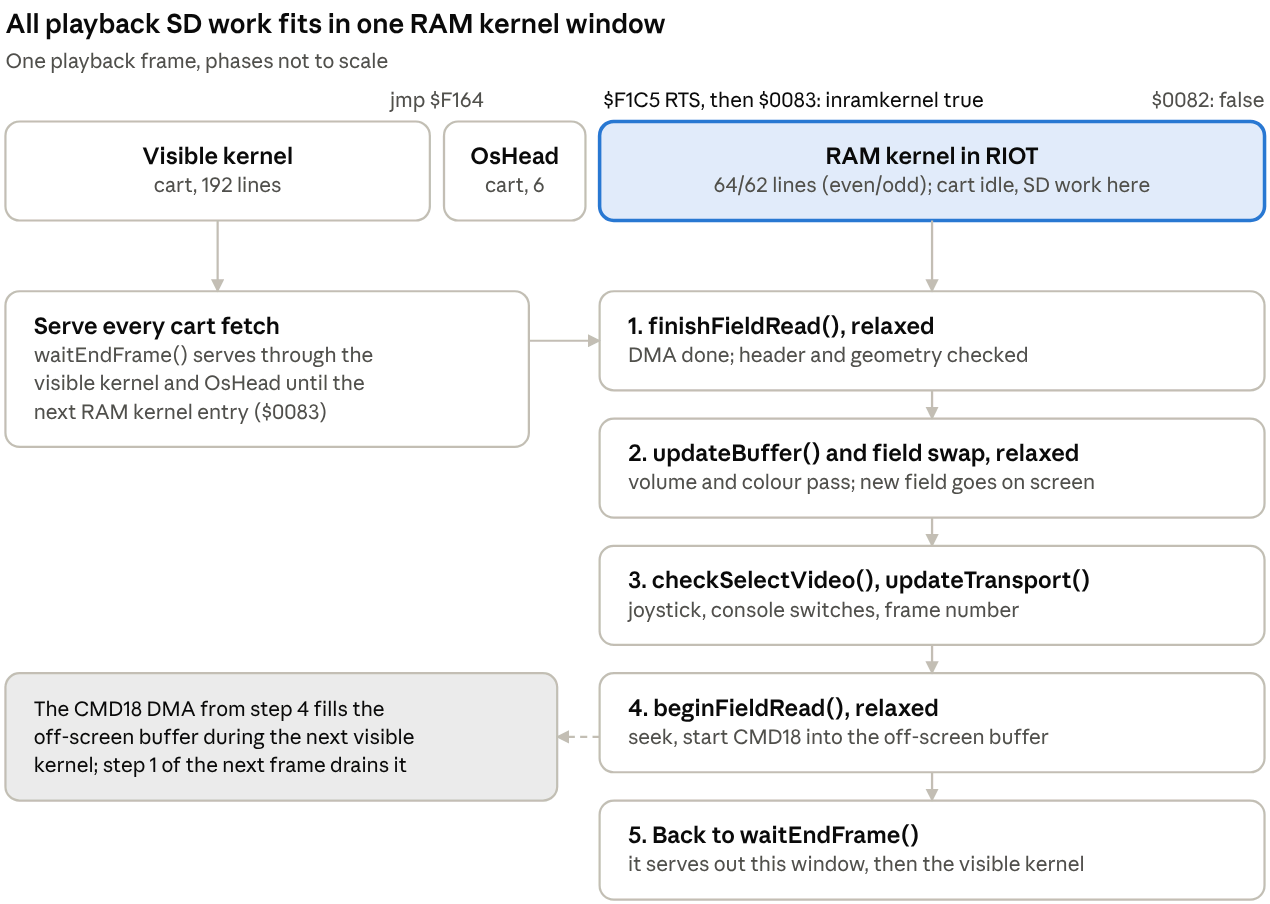
Milestones: 1 before SD init, 2 after mount, 3 after file open, 4 after the title probe, 5 at the start of playback. Failure codes used here: 5 mount failed, 6 no frame within 3 s of boot, 8 no movie file, 11 the movie's cluster chain could not be mapped, plus the `BLINK_*` codes for sector, header and geometry errors.

There is no kernel heartbeat any more: `diagFrameTick()` only ever set the LED off and was removed from `$F1C5`. The LED starts off and changes only when one of these routines drives it. `bus_serve_ms()` is now used only for these wall-clock codes.

Failure blinks still serve the bus (`bus_serve_ms()` serves every cart read and runs the tracker), so the console keeps displaying. Nothing swaps buffers, so the last field shown stays on screen and, after a failure during playback, its audio repeats as a 60 Hz buzz.

Frame reads and field swaps

During playback, every SD step for a field runs back to back inside one RAM kernel window in relaxed mode. The STM32 serves nothing from the `$0083` entry until `waitEndFrame()` serves again, before the RAM kernel exits at `$0082`.



one playback frame · 3 phases, 5 STM32 steps

The STM32 serves every cart fetch through the visible kernel and OsHead, runs steps 1–4 without touching the bus, and is serving again before the RAM kernel's `jsr VisibleBars`.

Playback (`runFrameLoop()`, `mc_playback_pipeline` = 1):

- Two field buffers (`mr_frameInfo1`, `mr_frameInfo2`). `beginFieldRead()` reads into `inactiveFrameInfo()`, the buffer whose visible use just ended; its DMA runs while the other buffer is on screen.
- Relaxed mode covers `pf_seek_block()` and `pf_read_blocks_begin()` in step 4, and in steps 1–2 `pf_read_blocks_finish()`, the `MVC\0` header and geometry checks, `updateBuffer()` and `mc_field_swap_to_display()`.
- The swap copies the new buffer's pointers and line counts into the frame the dispatch serves from, flips `mr_bufferIndex`, and packs the 32-byte blanking-audio tail that the next frame's OsHead serves at `$F140`.

- A failed read, or a CMD18 that fails to start, is retried into the same buffer (up to 4 attempts in all, logged as diagnostic code 5). A failed start is cleaned up by the driver and started again at the next RAM kernel entry. Either way that frame skips steps 2–4 and goes straight back to `waitEndFrame()`, so the previous field stays on screen. All attempts failing → `fatalBlink`.
- The first frame has no read pending, so it only starts one; the title shows once more.
- The swap in step 2 is the only one during playback. The frame waits' automatic swap (`mc_blanking_swap()`) is off while `mc_playback_pipeline` is set; if it ran, it could show the buffer still being filled by DMA, make `inactiveFrameInfo()` point the next DMA at the screen, or cancel step 2's swap. A frame without a successful load (the first frame, a retry, file select) shows the last field again.
- `pf_seek_block()` reads no FAT: it binary-searches the file's extent map (built at open) and computes the sector, a few microseconds whatever the jump. Rewinds, the Reset switch's 600-field jump and fast-forward all cost the same as normal play.

Title and NO_SD. Each frame wait (`emulate_cartridge()`, `moviecart_wait_ram_kernel_entry()`, `bus_wait_endframe()`, and yields outside relaxed mode) calls `mc_blanking_swap()` right after the entry. That alternates the two title buffers and repacks the blanking audio once per frame wait. `bus_serve_ms()` never swaps, and no frame wait swaps after ColdStart's entry.

Window time budget

A typical playback frame uses about 0.2–0.45 ms of the 3.95–4.08 ms RAM kernel window (62–64 lines, by field parity, from the `$0083` entry to the `$0082` exit), roughly 5–11%. These are estimates from cycle counts at 168 MHz and SDIO command timing at 8 MHz, 4-bit; nothing here has been measured.

Step	What it does	Typical	Worst (bounded)
Entry setup	`mc_on_ram_kernel_entry()`: pointers, line count	< 1 µs	< 1 µs
1. Finish the read	DMA/DATAEND flags, RXACT drain	~2 µs	—
	CMD12 stop	15–20 µs	resent while the card is still sending

Step	What it does	Typical	Worst (bounded)
	CMD13 status polls	1 poll, 15–20 µs	32 polls, ~0.6 ms
	Header check, `frameInit()`	< 2 µs	—
2. Process and swap	`updateVolume()`, 262 samples	25–45 µs	—
	`updateColor()`, 960 colour + 192 background bytes	100–300 µs	+20 µs with the overlay
	`mc_field_swap_to_display()` + audio pack	5–15 µs	—
3. Input	`checkSelectVideo()` (no Select), `updateTransport()`	< 5 µs	—
4. Start the next read	`pf_seek_block()` (extent map)	< 1 µs	< 1 µs
	DMA setup, data config, CMD18 + response	25–40 µs	—
Total		**0.2–0.45 ms (5–11%)**	**~1.1 ms (28%)**

Outside the window. The field's 5–6 sectors transfer by DMA during the visible kernel: card access latency (typically 0.1–1 ms) plus 0.64–0.77 ms at 4 MB/s, so the read normally finishes 1–2 ms after the CMD18, with about 12.5 ms to spare before step 1. FAT is never read during playback.

Bounded now. A read that has not ended is polled for at most 1 ms per window (`TM_FATFS_SD_SDIO_disk_read_done()`, no yields); after that the finish waits for the next RAM kernel entry, serving the bus while the DMA carries on and the shown field repeats. After 20 windows (~330 ms) the read is aborted and counts as a failed attempt. The card's own data timeout is now 250 ms (`SD_READ_DATATIMEOUT`) instead of `0xFFFFFFFF` clocks, so a stuck card normally ends the read with `SD_DATA_TIMEOUT` first. Relaxed work can no longer run past `$0082` waiting on the card.

Least certain. `updateColor()`'s real cost running from flash (the README estimates 250–300 µs, a cycle count gives about 100 µs) and each card's CMD12/CMD13 response times. A DWT timestamp at the entry and another before the next `waitEndFrame()`

would replace this table with measured numbers on the first hardware run.

Host test results

The firmware's real boot and playback code passes a full host run: 13,400 frames, every field shown identical to its original file, and every graphics, colour and audio write correct. `firmware/stm32/hosttest/run_tests.sh` compiles `main.c`, `core.c`, `pff.c`, `sd_reader.c`, `update.c` and `frame.c` for macOS, reads sectors from FAT32 disk images, and runs a cycle-stepped 6507 through the real boot ROM, visible kernel, OsHead and RAM kernel via the real `bus_dispatch()` and `mc_track_ram_kernel()`.

Area	Result
Field loading	Every field shown matches the original `.mvc` at the intended index, across 12 movies (NTSC, PAL, 4:3, both header formats)
Disks	FAT32 with and without an MBR partition, 4 KB and 32 KB clusters; two files fragmented into 55 cluster runs each
File filter	`README.TXT` and a macOS `._lion.mvc` AppleDouble file never picked
Transport	Play, pause, fast-forward (+5, +9, +13, +17…), rewind, Reset, Select, auto-advance and wrap-around all as designed
Fault recovery	A failed CMD18 start, a failed finish and a corrupted header each retried with no visible error
Served data	Every frame's GRP/COLUP/background writes, all AUDV0 writes, the audio pack after each swap, and input capture correct
Design rules	All field reads in relaxed mode; every SD access while the 6507 runs the RAM kernel; DMA targets aligned and off the stack
Frame timing	Every frame's line count matches the field parity (VBLANK 37/36, overscan 24/23); the RAM kernel plays 64 tail samples after an even field, 62 after an odd one; HMOVE always at cycles 2 and 72

Fixed after the first run. OsHead played the wrong audio samples: its loop's taken `bne` at `$F197` makes a dummy read of `$F199`, which ran the `AUD5` handler and consumed a sample on 4 of its 6 lines; the pointer now advances on the operand fetch instead. OsHead's first line also overran by 4 cycles (it starts at cycle 71), taking 7 lines instead of 6; its setup is now 10 cycles at `$F164` (`ldx #0` / `stx GRP1` / `stx GRP0` / `ldy #5`), and the visible kernel jumps there.

Interlace and VBLANK fixed. The RAM kernel now sets its line counts from the parity of the field just shown, as the PIC does: OsHead loads `frameInfo.odd` into RIOT `$FB` (`lda #odd` at `$F1BF`, `sta $FB`), and the kernel runs 24 overscan + 37 VBLANK lines after an

even field, 23 + 36 after an odd one. With OsHead's 6 lines and the pad line that is 263 then 261, so the picture alternates by one line and every VSYNC-to-VSYNC frame is 262 lines when fields alternate. The RAM kernel grew to 82 bytes (`$80` – `$D1`), the audio pack moved to `$D8` – `$F7` , and OsHead's RTS moved to `$F1C5` . In the 4 KB run 5,001 frames were 262 lines; the 1,124 at 261 or 263 are frames whose parity repeated the previous one (title, pause, rewind steps, file changes). PAL fields (311/312) use the same counts.

Startup. Milestone blinks 1–5 take 1,215 frames, about 20 s, before the first movie field appears. `make NO_LED=1` now drops them, and the title keeps alternating fields through blink 5 (the playback pipeline flag is set after it).

Second independent audit

A fresh review found no fault in the 6502 code, the interlace, the field pipeline or transport. It found three problems worth fixing, now fixed and covered by the host test; the rest are low severity and still open.

Finding	Severity	Status
SD read finish could poll for minutes in relaxed mode; a card stall over ~4 ms crashed the 6507 with no blink	High	Fixed: 1 ms per window, 20 windows, then abort and retry; card data timeout 250 ms. Host test injects a read taking 3 frames and one that never ends
File change played ~1.3 s of 60 Hz buzz (zeroed field audio, stale blanking pack)	Medium	Fixed: `mc_mute_display()` silences the shown field and the pack as soon as Select or end of file is seen. Host test now fails any 3-frame buzz run (it caught 7 without the fix)
Milestone blinks add ~20 s of boot, ignore `NO_LED` ; title stops interlacing during blink 5	Medium	Fixed: `NO_LED=1` drops them; pipeline flag set after blink 5
CMD12 is sent a frame after the data, so a field in the last ~64 KB of the card could get OUT_OF_RANGE	Low–medium	Open (plausible, untested). Separately, an abort now waits (bounded) for the DMA stream to stop before a retry reconfigures it

Finding	Severity	Status
Clusters under 4 KB can never play (blink 10)	Low	Open
`visibleLines` under 22 corrupts memory in the overlay	Low	Open (PIC has it too)
RAM kernel ignores the header's overscan/VSYNC/VBLANK counts; overscan is not VBLANKed	Low	Open (all example files use 3/37/30)
Select loops forever if file 1 stops opening; one-field file goes to frame −1; failed directory reads unchecked	Low	Open

Assumptions and open risks

The design rests on one assumption: relaxed work always finishes before the RAM kernel's `jsr VisibleBars`, 62–64 lines (roughly 4 ms) after the `$0083` entry. None of this has been run on hardware.

- **Late DMA.** Fixed: the finish waits at most 1 ms per window and 20 windows, then aborts and retries (see Window timing).
- **Fixed after an independent review.** Card addresses at or past 4 GiB no longer wrap (`(uint64_t)sector << 9` in the SDIO driver). A failed CMD18 start now cleans up the data path (`TM_FATFS_SD_SDIO_disk_read_abort()`) and counts as one of the field's four attempts instead of being fatal. The card-status wait after a read is bounded to 32 CMD13 polls and resends CMD12 if the card is still sending. The seek table was then replaced by the extent map, which takes FAT reads out of playback entirely.
- **Open from the same review.** A `.MVC` file of 4–8 KB makes the end-of-file clamp go to frame −1; it stops at Select with blink 7 on 32 KB clusters (the probe reads past the end of the file) or blink 9 on 4 KB clusters (the seek is refused), instead of hanging. `visibleLines` under 22 lets the overlay write before the field buffer. Suspected only: CMD12 now comes a frame after the data, so a field near the end of the card could get OUT_OF_RANGE; a CRC error on the last block can be dropped (stock ST code).
- **FAT lookups.** None during playback. When a movie is opened (at boot and on Select or auto-advance), `pf_map_file()` reads the FAT six sectors per frame with one async CMD18, started in one RAM kernel window and drained in the next like a field read, and

records the file's clusters as runs of consecutive clusters (up to 8,192 runs, 64 KB of SRAM1). About one frame per 768 clusters: roughly 0.6 s for an hour of video on 32 KB clusters, 5 s on 4 KB clusters, up to ~23 s for a 4 GB file on 4 KB clusters. A broken chain or more than 8,192 fragments stops with blink 11; a FAT read that fails 4 times, blink 10.

- **Stale flag.** `inramkernel` only changes while a bus loop runs. It stays true through relaxed work, which is correct only while that work ends inside the window.
- **Delays.** `moviecart_delay_ms()` waits for a fresh RAM kernel entry on every pass, relaxed or not, so it always serves the bus; it lasts at least one frame.
- **Slow non-relaxed paths.** Every yield outside relaxed mode costs a frame: SD init, mount and file open take at least two frames per SDIO command, and file select takes about 2 s or more.
- **Frozen picture during a failure code.** The console keeps displaying, but the last field stays on screen and, after a failure during playback, it is now silent: fatalBlink() calls mc_mute_display() before blinking.
- **Build.** The changes pass a clang syntax check, including the NO_SD, `INIT_PHASE=3` and `STALL_TEST` variants, but have not been built with `arm-none-eabi-gcc`.
- **Stale comment.** `src/defines.h:25` still says `POLL_READ` drains the SDIO FIFO in the pump's free cycles; no code does that.
- **DMA contention.** The field DMA writes SRAM while `bus_dispatch()` reads the other buffer through the visible kernel. The design was already like this; its effect on the ~100 ns per-cycle budget has not been measured.
- **Cards and files that stop playback.** Clusters smaller than 4 KB make a 5–6-sector field cross a cluster, which `pf_read_blocks_begin()` refuses, so the first field read ends in `fatalBlink`. If no movie file opens on Select, `checkSelectVideo()` retries file 1 forever (serving the bus, but never resuming). Non-movie files in the root are now skipped (see Opening the file); a movie file whose name starts with `_` is skipped too.
- **Unread diagnostics.** `mc_diag_note()` still records codes such as `DIAG_FIELD_RETRY` and the visible line-count wrap, but nothing reads them out since the heartbeat was removed, so a recovered field retry is invisible.
- **Fast-forward.** Not a separate risk: `MAX_SPEED` in `update.c` caps `uInfo.speed` at 125 fields per frame, and with the extent map a jump of any size costs the same few microseconds as normal play.